JobForge: Distributed Job Scheduler

Submission Links

- **GitHub Profile**: https://github.com/Spiderboyis
- **GitHub Repository**: https://github.com/Spiderboyis/distributed-job-scheduler
- **Frontend App (Vercel)**: https://distributed-job-scheduler-frontend-six.vercel.app/
- **Backend API & Worker (Render)**: https://jobscheduler-backend.onrender.com

---

1. Setup & Deployment Instructions

1.1 Prerequisites

- Node.js (v24.x)
- PostgreSQL (v15+)
- Docker & Docker Compose
- Git

1.2 Local Setup (Development)

1. Clone the repository and install workspace dependencies:

```
git clone https://github.com/Spiderboyis/distributed-job-scheduler.git
cd distributed-job-scheduler
npm install
```

2. Configure Environment Variables: Create a `.env` file in `packages/backend` :

```
PORT=3001
DATABASE_URL=postgresql://jobscheduler:jobscheduler_secret@localhost:5432/jobscheduler
JWT_SECRET=your_super_secret_key
JWT_REFRESH_SECRET=your_refresh_secret
NODE_ENV=development
```

3. Database Migration & Seeding:

```
# Run PostgreSQL migrations
npm run db:migrate --workspace=packages/backend

# (Optional) Seed the database with demo data
npm run db:seed --workspace=packages/backend
```

4. Start the Application:

```
# Terminal 1: Backend
npm run dev:backend
```

```
# Terminal 2: Frontend
npm run dev:frontend
```

1.3 Docker & Docker Compose Setup

To launch the complete PostgreSQL Database, Express.js Backend, and Next.js Frontend stack inside isolated Docker containers, simply run:

```
docker compose up --build
```

This automatically builds the containers, runs migrations, and links the services together. The services will be accessible at:

- **Frontend Panel**: `http://localhost:3000`
- **Backend API**: `http://localhost:3001`
- **PostgreSQL Database**: `localhost:5432` (credentials: `jobscheduler` / `jobscheduler_secret`)

1.4 Production Deployment (Render / Cloud)

This application is configured for direct cloud deployment using services like Render, Vercel, and cloud PostgreSQL databases.

Database Deployment (PostgreSQL):

- **Platform**: Managed production-grade PostgreSQL (v15+) instance hosted on **Render PostgreSQL** (or equivalent cloud providers like Supabase, Neon, or AWS RDS).
- **Configuration**: Handles concurrent connections from distributed workers via connection pooling.
- **Trigger Triggers**: Relies on PostgreSQL trigger mechanisms and `LISTEN` / `NOTIFY` to stream real-time updates (through Server-Sent Events) to the React dashboard.
- **Migrations**: Automated migrations run dynamically prior to service startup using the connection string configured under `DATABASE_URL`.

Backend & Worker Deployment:

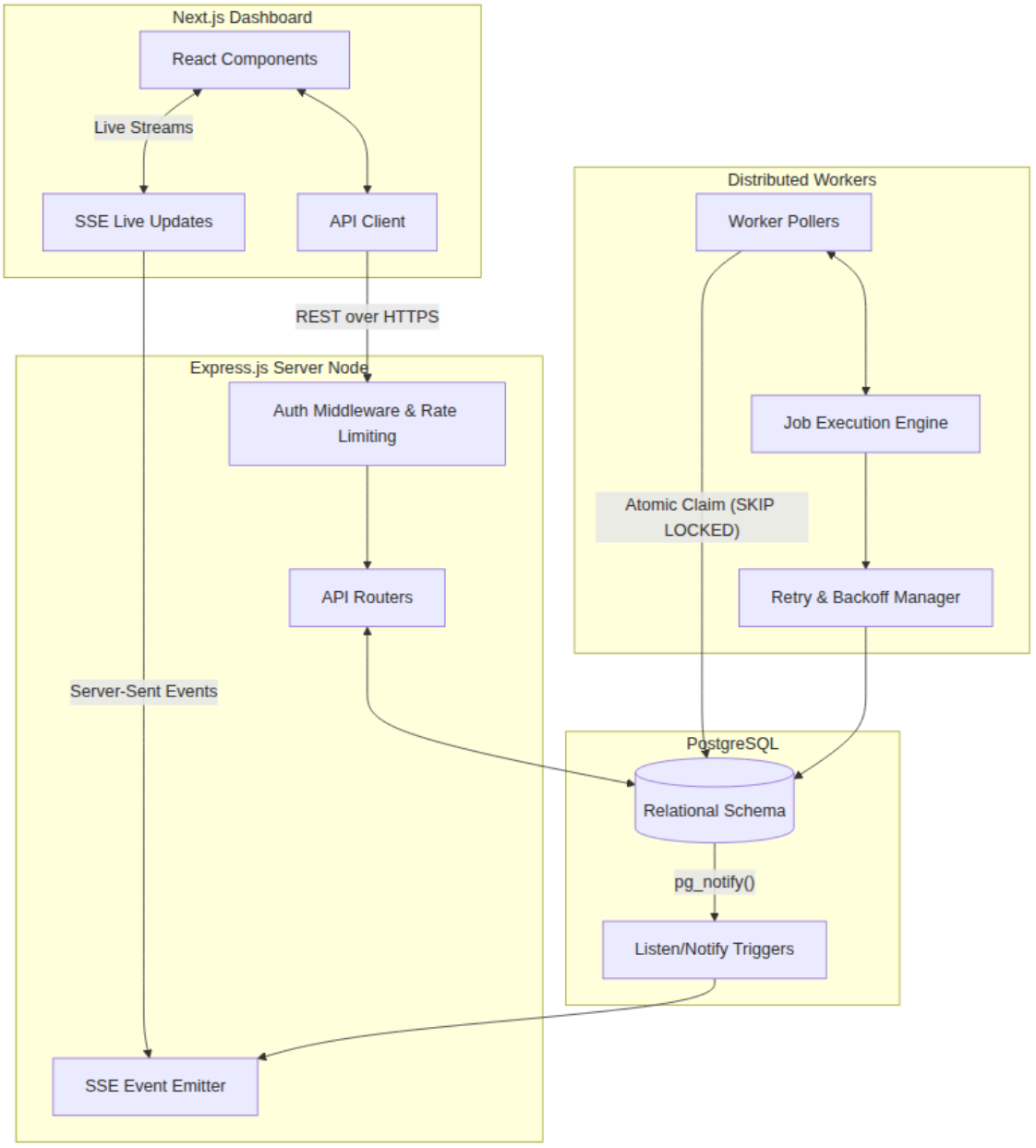
- **Host**: **Render** (Web Service & Cron / Background Workers)
- **Build Command**: `npm install --include=dev && npm run build:backend`
- **Start Command**: `npm run db:migrate && node packages/backend/dist/index.js`
- **Port**: `3001`
- **Environment Variables**:
  - `DATABASE_URL`: Connection string pointing to your managed cloud PostgreSQL database.
  - `JWT_SECRET` & `JWT_REFRESH_SECRET`: Secure cryptographic keys.
  - `NODE_ENV`: `production`

Frontend Deployment:

- **Host**: **Vercel**
- **Build Command**: `npm install --include=dev && npm run build:frontend`
- **Start Command**: `npm run start --workspace=packages/frontend`
- **Port**: `3000`
- **Environment Variables**:
  - `NEXT_PUBLIC_API_URL`: `https://jobscheduler-backend.onrender.com` (Public address of your deployed Backend API on Render).

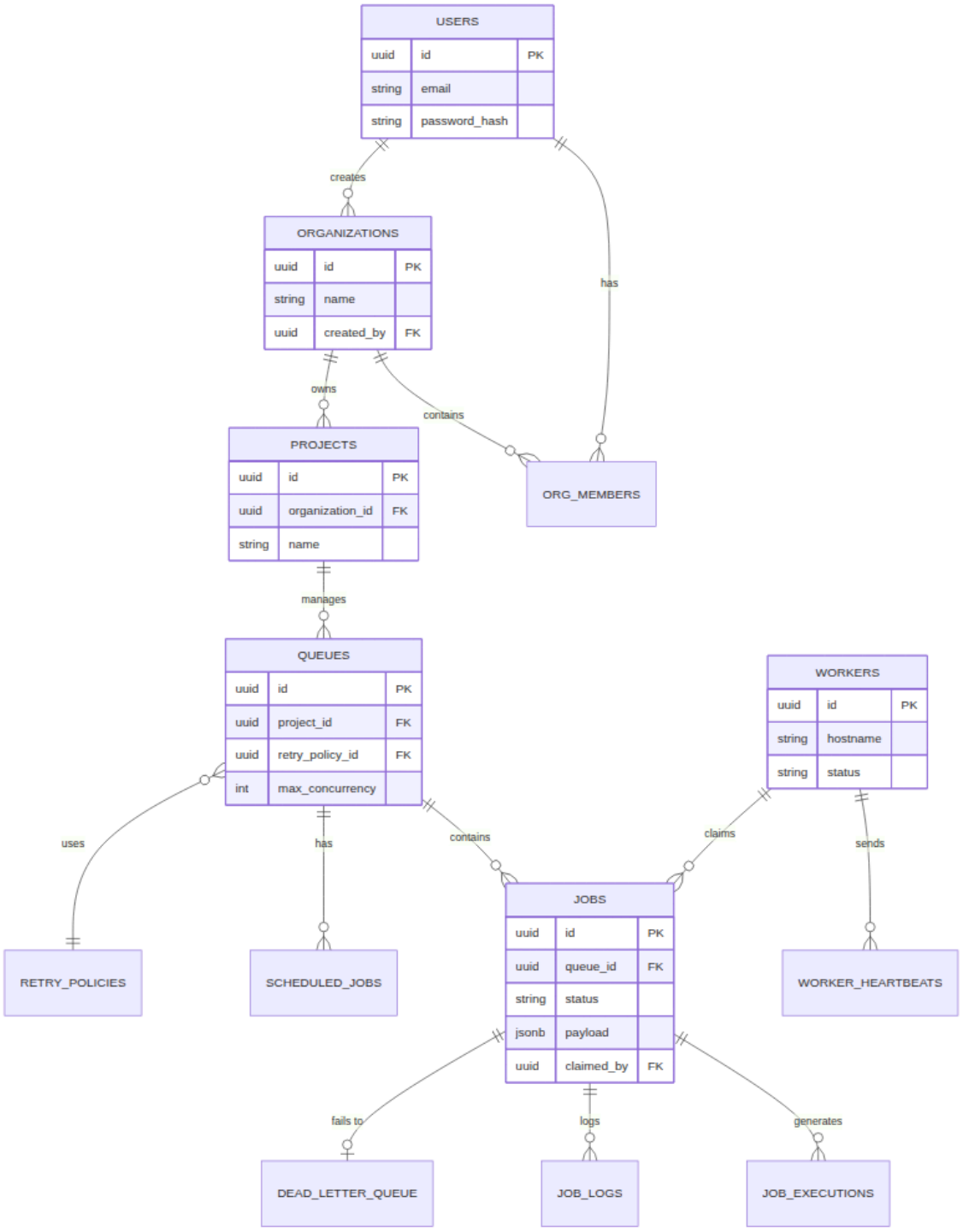
---

2. Architecture Diagram



---

3. Entity Relationship (ER) Diagram



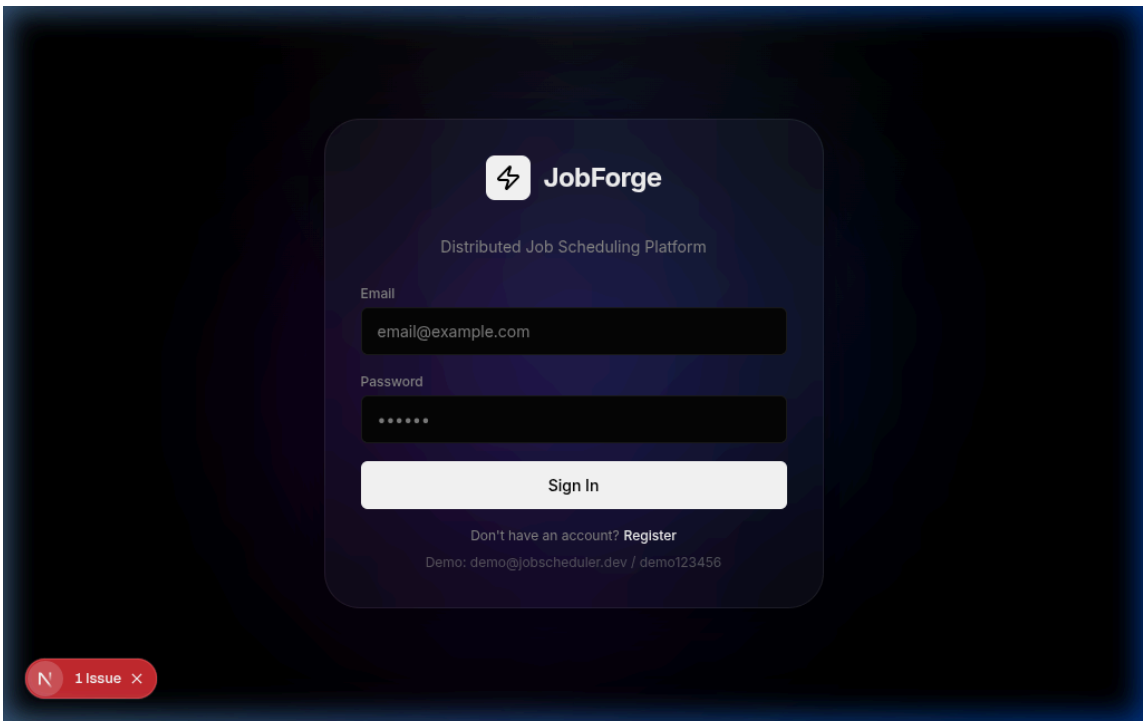
---

4. Visual Deliverables & UI Screenshots

4.1 Login Page & Glassmorphism Design

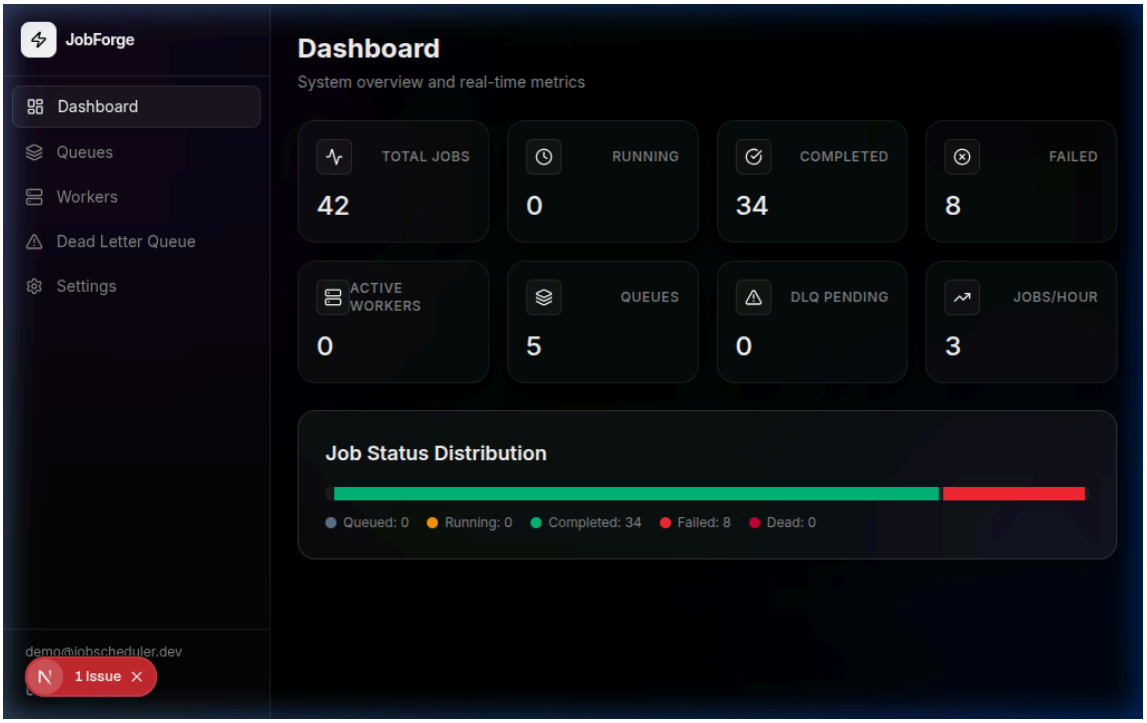
The login page has been rewritten with a modern iOS-inspired glassmorphism layout, featuring floating blurred glowing orbs in the background and a sleek, authentic card design. Default demo credentials are

removed from state but preserved as a small help hint.



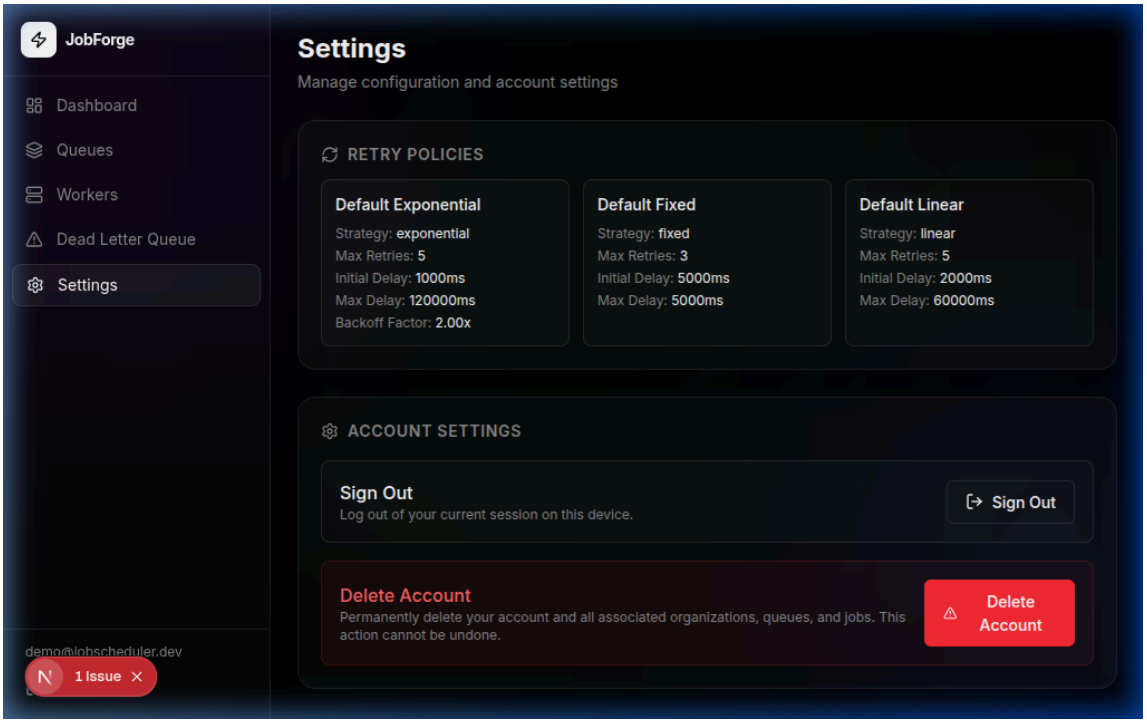
4.2 Dashboard & Tenant Isolation

All database select operations have been restricted using PostgreSQL joins against the `org_members` table. When logging in as a newly created user, the metrics, queues, and active worker stats show exactly `0` values rather than displaying global demo queue stats, verifying airtight tenant isolation.



4.3 Settings Page

The Settings tab has been streamlined to highlight retry policies, simple sign-out action, and a secure account deletion flow requiring password validation.



---

5. API Documentation

Authentication & Account

- `POST /api/auth/register` : Register a new user.
- `POST /api/auth/login` : Authenticate and receive a JWT.
- `DELETE /api/auth/account` : Securely delete the account and securely cascade-delete all owned resources (organizations, queues, jobs).

Queues

- `GET /api/projects/:projectId/queues` : List all queues in a project.
- `POST /api/projects/:projectId/queues` : Create a new queue with specific concurrency and retry policies.
- `POST /api/queues/:queueId/pause` / `resume` : Halt or resume job processing for a queue.
- `GET /api/queues/:queueId/stats` : Fetch real-time metrics (throughput, failure rates) for a queue.

Jobs

- `POST /api/queues/:queueId/jobs` : Enqueue a new immediate, delayed, or scheduled job.
- `POST /api/queues/:queueId/jobs/batch` : Atomically enqueue up to 100 jobs at once.
- `GET /api/queues/:queueId/jobs` : Fetch paginated jobs with status filtering.
- `GET /api/jobs/:jobId` : Get detailed execution history and logs for a specific job.
- `POST /api/jobs/:jobId/retry` : Manually requeue a failed or dead job.

Live Metrics (SSE)

- `GET /api/sse/events?token={JWT}` : Streams real-time, tenant-isolated dashboard metrics via Server-Sent Events.

---

6. Design Decisions & Trade-offs

1. Database as the Message Queue

Decision: We utilized PostgreSQL to handle job queuing rather than introducing a specialized broker like Redis or RabbitMQ. **Trade-off:** While Redis offers superior pure in-memory throughput, utilizing Postgres with `FOR UPDATE SKIP LOCKED` provides robust, atomic job claiming. This significantly reduces infrastructure complexity, ensures strict ACID compliance, and inherently solves the "two-phase commit" problem where database state and queue state become misaligned.

2. Event-Driven Live Updates (SSE vs WebSockets)

Decision: We chose Server-Sent Events (SSE) over WebSockets for dashboard live updates, powered by Postgres `LISTEN/NOTIFY` . **Trade-off:** WebSockets provide bi-directional communication, but our dashboard only requires unidirectional data flow (server-to-client). SSE is natively supported by HTTP/1.1, avoids complex handshake overhead, automatically handles reconnections, and seamlessly propagates database triggers to the React frontend.

3. Tenant Data Isolation

Decision: All resources structurally cascade from an `Organization` , and security is enforced at the SQL level via explicit `JOIN` s against the `org_members` table on every backend route. **Trade-off:** This introduces slight overhead on read queries due to multi-table joins. However, it guarantees airtight multi-tenancy. A user

completely dropping their account will safely `CASCADE DELETE` all their data without risking global data corruption, which is a critical requirement for a distributed SaaS environment.

4. UI/UX: High-Fidelity Glassmorphism

Decision: We bypassed generic UI libraries in favor of a custom, heavily polished "Midnight Minimalist" aesthetic featuring dynamic background orbs and deep `backdrop-blur-2xl` glass cards. **Trade-off:** It takes significantly more CSS optimization (especially performance tuning for blurs) than importing a standard Tailwind UI kit. However, it successfully delivers a highly premium, state-of-the-art "Wow" factor requested for modern developer tooling.

---

7. Core Platform Walkthrough & Functional Flow

This section details the step-by-step user journey and technical pipeline as jobs progress through the scheduler system:

1. **User Registration & Token Handshake**:
    - The user visits the Landing page (`/`) and creates a new account.
    - Upon submitting details, `POST /api/auth/register` creates the user record and automatically establishes a default tenant `Organization` and `Project` linked to them.
    - Logging in via `POST /api/auth/login` verifies credentials against `bcrypt` hashes and returns a signed JSON Web Token (JWT) containing user and role properties. This token is saved in client storage for authenticated header verification on all subsequent requests.
2. **Creating and Configuring Queues**:
    - Within the dashboard view (`/dashboard/queues`), the operator clicks "+ New Queue".
    - The form allows specifying key configuration bounds: Queue Name, Concurrency Limit (maximum parallel active jobs), Priority Rating, and a default Retry Policy (Fixed delay, Linear backoff, or Exponential backoff).
    - Firing the form invokes `POST /api/projects/:projectId/queues` which writes to the database. The system automatically scopes this request, preventing a user from modifying queues belonging to other organizations.
3. **Job Ingestion & Timing Configuration**:
    - Within a queue detail page (`/dashboard/queues/[queueId]`), users click "+ New Job" to enque job payloads.
    - The scheduler supports three timing configurations:
        - **Immediate Execution**: Sets the execution timing field (`run_at`) to the current timestamp (`NOW()`) and status to `queued` .
        - **Delayed Execution**: Delays starting by a user-specified delay duration (in seconds), setting `run_at` to `NOW() + delay` and status to `scheduled` .
        - **Scheduled Execution**: Sets `run_at` to a specific future date-time calendar picker timestamp and status to `scheduled` .
    - The payload input allows submitting structured JSON objects (e.g. `{"userId": 100, "action": "send_welcome"}`). This calls `POST /api/queues/:queueId/jobs` to atomically persist the job.
4. **Distributed Worker Claiming Loop**:

- The independent worker service (`packages/backend/src/worker/worker.ts`) registers itself in the database and enters a poll loop.
- The worker targets eligible jobs whose `run_at <= NOW()` and whose status is currently `queued` or `scheduled` .
- To prevent multiple workers from executing the same task, the worker performs atomic claims using a PostgreSQL query with the `SELECT ... FOR UPDATE SKIP LOCKED` clause. This locks and updates claimed jobs to `status = 'claimed'` instantly while bypassing already-locked rows, ensuring linear and distributed concurrency without race conditions.

5. **Execution Simulation & Logs**:

- Once claimed, the worker transitions the job status to `running` , starts a timer, and spins up a simulated processing workload corresponding to the job type (e.g., resizing images or compiling pdf reports).
- An execution audit record is written to the `job_executions` table detailing the assigned worker's hostname, process ID, execution state, and exact timestamp.
- Upon successful execution, the job status is set to `completed` and final metrics are saved.

6. **Error Handlers & Retry Policies**:

- If job execution encounters an error (e.g., simulated network timeout or system overload), the worker catches the exception and reviews the queue's retry policy.
- If the current attempts have not reached the limit (`attempts < max_retries`), the system calculates the backoff interval (e.g., exponential delay). The job is moved back to the `queued` state, and its `run_at` timestamp is pushed forward by the calculated delay.
- If attempts exceed the limit, the job is labeled as `failed` , ejected from processing queue, and sent to the **Dead Letter Queue (DLQ)**.

7. **Dead Letter Queue (DLQ) & Operator Recovery**:

- Administrators can monitor failing nodes inside the Dead Letter Queue dashboard view (`/dashboard/dlq`).
- The DLQ displays critical diagnostics: the error message stack trace, the source queue, total retry attempts, and the failure timestamp.
- Operators can rectify downstream issues and trigger a manual retry. Clicking "Retry" sends a `POST /api/jobs/:jobId/retry` request to reset the attempts to `0` and place the job back in the `queued` pipeline with `run_at = NOW()` .

8. **Live Dashboard Telemetry (SSE)**:

- When the user views the main Dashboard page, the browser starts a Server-Sent Events stream: `/api/sse/events?token={JWT}` .
- The backend validates the token, listens for database events via PostgreSQL's built-in `LISTEN/NOTIFY` protocol, and broadcasts real-time updates of job metrics and active worker heartbeats over a single HTTP connection. The client instantly refreshes without page reloads.

9. **Secure Account & Workspace Purging**:

- If a user chooses to delete their account under Settings (`/dashboard/settings`), they enter their password for authentication verification.
- The backend fires `DELETE /api/auth/account` . This executes a database transaction that deletes the user record and triggers a cascading cascade deletion of organizations

created/owned by the user. All associated projects, queues, jobs, and execution logs are deleted cleanly from the database, while leaving data of other platform tenants untouched.